

Assignment-3

Server Side Web Development with Node.js(CSE 3192)

Semester: 6th

1. Write a Node.js program that: reads a file provided as a command-line argument, compresses the file using gzip, saves the compressed file with .gz extension and displays a success message after completion
2. Write a Node.js program that: Reads a file using createReadStream() and subsequently Compresses the file using zlib.createGzip() and writes the compressed data to a new file using createWriteStream() and finally displays a message when compression is complete.
3. Analyze how backpressure is handled in the given Node.js server using res.write() and the 'drain' event. Explain the role of the return value of write() (shouldContinue) in controlling data flow. Discuss how this mechanism prevents buffer overflow and ensures efficient streaming.
4. Apply your knowledge of Node.js streams to develop a server-side program that: (a)Creates an HTTP server using createServer(), (b) Receives a compressed file from a client request, (c) Extracts the filename from request headers (x-filename), (d) Decompresses the file using createGunzip(), (e)Saves the decompressed file into a folder named received_files , (f)Sends a response message **"OK"** after successful file storage.
5. Develop a Node.js client-side application that applies stream processing and HTTP communication concepts to transfer a file to a remote server. The program should accept a filename and server host as command-line arguments, read the specified file using a readable stream, compress the file data using gzip, and send the compressed content to an HTTP server using the PUT method. Appropriate HTTP headers such as Content-Type, Content-Encoding, and X-Filename must be included in the request. After sending the file, the program should display the server's response status code and print a confirmation message indicating that the file has been successfully transmitted.
6. A Node.js program creates a readable stream from the following structured dataset using Readable.from() and processes it using the 'data' event to display formatted output:

```
const mountains = [  
  { name: 'Everest', height: 8848 },  
  { name: 'K2', height: 8611 },  
  { name: 'Kangchenjunga', height: 8586 },  
  { name: 'Lhotse', height: 8516 },  
  { name: 'Makalu', height: 8481 }  
]
```

Evaluate the effectiveness of using Readable streams for processing this dataset. Critically compare this approach with traditional iteration methods such as for, forEach, or map in terms of performance, scalability, memory usage, and real-time data

handling. Justify whether streams are suitable for handling such in-memory collections and recommend improvements or alternative techniques where necessary.

7. Design and develop a Node.js function that concatenates multiple input files into a single destination file using stream-based processing. The function should accept a destination filename and an array of source filenames, create a writable stream for the destination, and use `Readable.from()` along with a custom Transform stream in object mode to sequentially read and pipe each source file into the destination without closing the stream prematurely. Ensure proper handling of stream events such as 'error', 'end', and 'finish', and return a Promise that resolves upon successful completion of the concatenation process. Additionally, enhance the design by suggesting improvements for efficiency, scalability, or error handling.